

# Tensor Memory: Fixed-Size Recurrent State for Long-Horizon Transformers

Kabir Swain<sup>1</sup> Sijie Han<sup>2</sup> Daniel Karl I. Weidele<sup>3</sup> Mauro Martino<sup>3</sup> Antonio Torralba<sup>1</sup>

## Abstract

Transformers process images and videos by flattening space and time into long token sequences. While attention and KV caching preserve past features, their memory grows with sequence length and they lack an explicit, persistent spatial state, making long-horizon video understanding and occlusion-sensitive reasoning difficult. We propose Tensor Memory, a lightweight module that augments Transformer blocks with a fixed-size recurrent 3D memory tensor: tokens write into a voxel grid via a differentiable soft write that deposits content as a Gaussian-weighted volume around a predicted continuous 3D location, the memory is updated with an efficient local interaction operator and gated recurrent dynamics, and tokens read back context via continuous sampling with gated residual fusion. Because the memory tensor has a constant size, Tensor Memory decouples state capacity from input length while preserving a spatial inductive bias. We evaluate the module on standard language, image, and video benchmarks and on a controlled toy diagnostic suite designed to isolate when persistent state is beneficial; it integrates with standard Transformer training pipelines and can be attached to or removed from existing blocks without other architectural changes.

## 1. Introduction

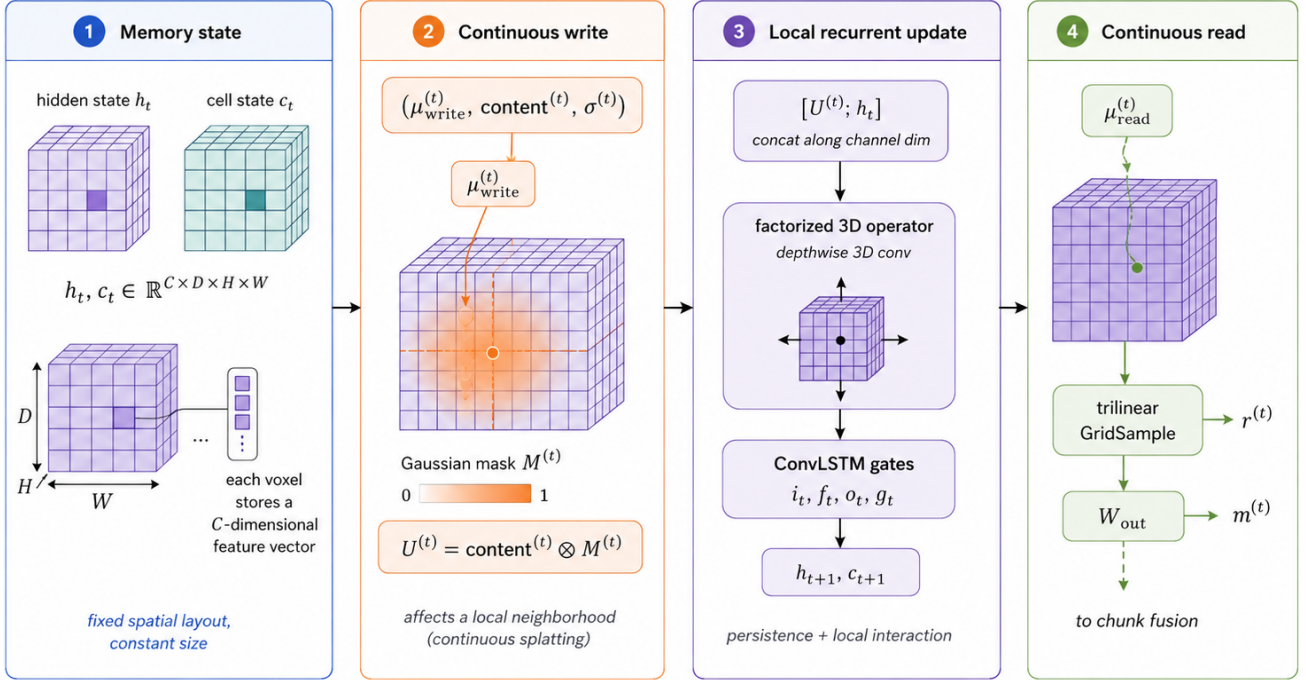
Transformers are now the standard model across language and vision because attention can flexibly combine information over tokens (Vaswani et al., 2017). Figure 1 gives an overview of Tensor Memory: tokens read from and write to a fixed-size 3D voxel state, the voxel state is updated recur-

rently with lightweight dynamics, and the retrieved memory signal is fused back into the token stream through a gated residual path. At the same time, most modern pipelines reduce everything to a single token stream. Text is naturally sequential, but images are split into patches and flattened (Dosovitskiy et al., 2020; Touvron et al., 2020), and videos become extremely long sequences of spatiotemporal tokens (Bertasius et al., 2021; Liu et al., 2021). This framing works well at moderate lengths, but it exposes two weaknesses when the horizon grows. First, practical inference relies heavily on storing past keys and values, which scales linearly with the number of tokens and becomes costly for long videos and long documents (Kwon et al., 2023; Shazeer, 2019; Ainslie et al., 2023). Second, a vanilla Transformer does not maintain an explicit persistent state. It can attend back to past features, but it does not keep a structured representation that can be updated and queried as the input changes. In vision, this often shows up as fragility under occlusion or missing observations, and in long-horizon settings, it means repeatedly reconstructing global context from an ever-growing history rather than maintaining a compact state that persists.

Prior work addresses long sequences through sparse/structured attention (Beltagy et al., 2020; Zahoor et al., 2020; Kitaev et al., 2020), history compression or recurrence (Dai et al., 2019; Rae et al., 2019), learned memory mechanisms (Bulatov et al., 2022; Jaegle et al., 2021), and retrieval-augmented context (Khandelwal et al., 2019; Borgeaud et al., 2021; Lewis et al., 2020). These methods can extend context length, but the stored information is usually still an unstructured collection of token vectors or slots. For tasks that are naturally state-based, especially in video and embodied settings, the more relevant question is not only what was seen, but what is currently believed to be true about the scene. Separately, voxel grids and feature volumes provide strong spatial structure in vision, but they are typically used as intermediate representations rather than as a reusable memory that tokens can write to and read from over time. This motivates a memory mechanism that stays fixed in size, preserves spatial structure explicitly, and integrates cleanly into standard Transformer backbones.

We introduce Tensor Memory, a lightweight module that augments Transformer blocks with a fixed-size recurrent

<sup>1</sup>Massachusetts Institute of Technology, Cambridge, MA, USA <sup>2</sup>University of Toronto, Toronto, Canada <sup>3</sup>IBM Research, Cambridge, MA, USA. Correspondence to: Kabir Swain <kswain@mit.edu>, Sijie Han <hs.han@mail.utoronto.ca>, Daniel Karl I. Weidele <daniel.karl@ibm.com>, Mauro Martino <mmartino@us.ibm.com>, Antonio Torralba <torralba@mit.edu>.



**Figure 1. Tensor Memory: a fixed-size 3D voxel state with continuous writes, local recurrent updates, and continuous reads.** (1) The memory is a pair of volumes  $h_t, c_t \in \mathbb{R}^{C \times D \times H \times W}$  (hidden and cell states); each of the  $D \times H \times W$  voxels stores a  $C$ -dimensional feature vector and the spatial layout is fixed and constant in input length. (2) Each step  $t$  emits a write package  $(\mu_{\text{write}}^{(t)}, \text{content}^{(t)}, \sigma^{(t)})$  that deposits a Gaussian-weighted volume  $U^{(t)} = \text{content}^{(t)} \otimes M^{(t)}$  around  $\mu_{\text{write}}^{(t)}$ , affecting a local neighbourhood through a smooth mask  $M^{(t)}$  (continuous splatting). (3) Concatenating the write volume with the current state  $[U^{(t)}; h_t]$  along the channel axis, a factorized 3D operator (axis-aligned depthwise 3D convolutions plus a pointwise mix) produces ConvLSTM gates  $(i_t, f_t, o_t, g_t)$  that yield the next state  $(h_{t+1}, c_{t+1})$ , giving persistence and local voxel-to-voxel interaction. (4) A predicted read coordinate  $\mu_{\text{read}}^{(t)}$  trilinearly samples  $h_t$  to obtain  $r^{(t)}$ ; a learned projection  $W_{\text{out}}$  produces the readout  $m^{(t)}$  that is fused back into the chunk through a gated residual.

3D memory tensor that acts as an explicit spatial state. The memory is a small voxel grid with  $C$  channels whose size is constant with respect to input length. Each layer applies a simple cycle: tokens write information into the grid by predicting what to store and a continuous 3D location, then depositing content as a differentiable, Gaussian-weighted volume around that location to avoid hard voxel assignment and to allow gradients to shape where information is placed. The memory state is then updated through an efficient local interaction operator together with gated recurrent dynamics, allowing information to persist, propagate locally, and remain available when observations are missing or corrupted. Tokens read back context by predicting a query coordinate and using trilinear sampling (Jaderberg et al., 2015), and the retrieved features are fused back into the token stream through a gated residual connection with a learned scalar gate, making the module stable to attach to pretrained models.

Although the design is motivated by long-horizon video, the same mechanism is useful beyond video. For language, Tensor Memory provides a compact recurrent state that can accumulate global context without growing a token-level

cache. For images, it serves as a global workspace that encourages structured aggregation and can help on problems where global spatial composition matters. In this paper, we describe the architecture in detail and evaluate it across language modeling on WikiText-2, image patch reconstruction on CUB-200-2011, video action recognition on UCF-101, and a controlled toy diagnostic suite designed to isolate when persistent state helps.

## 2. Related Work

Most work on long sequences in Transformers focuses on making attention cheaper rather than changing what “memory” represents. Segment recurrence in Transformer-XL extends context by reusing past activations across segments (Dai et al., 2019), and Compressive Transformers summarize older states into a smaller memory to push the horizon further without keeping the entire history at full resolution (Rae et al., 2019). A large body of efficient-attention research reduces quadratic cost through three primary strategies: sparse or structured attention patterns like Longformer (Beltagy et al., 2020) and BigBird (Zaheer et al., 2020), low-rank projections such as Linformer (Wang et al., 2020),

and kernel-based linear attention like Performer (Choromanski et al., 2020). In parallel, IO-aware exact attention kernels such as FlashAttention improve practical memory/throughput without changing the attention interface (Dao et al., 2022).

More recently, large-scale LLM systems have emphasized reducing inference-time KV-cache cost while keeping attention as the interface. For example, DeepSeek-V3 adopts Multi-head Latent Attention to improve inference efficiency by compressing the information stored for attention (Liu et al., 2024). Related approaches reduce KV memory by sharing or grouping key/value heads, e.g., multi-query attention (MQA) (Shazeer, 2019) and grouped-query attention (GQA) (Ainslie et al., 2023), while systems work such as vLLM manages KV allocation efficiently via paging-inspired mechanisms (PagedAttention) (Kwon et al., 2023). Other recent long-context strategies retain only a window of recent KV states while preserving a small set of “sink” tokens to stabilize attention (Xiao et al., 2023), or incorporate bounded compressive memory directly into the attention mechanism (Infini-attention) (Munkhdalai et al., 2024). Closest in spirit to replacing an ever-growing KV cache, Trellis learns a test-time compression mechanism that maintains a fixed-size key-value memory (Karami et al., 2025).

A separate line of work adds explicit external memory. Neural Turing Machines introduced differentiable read/write access to an external memory matrix (Graves et al., 2014). Recent work from DeepSeek proposes a different memory primitive aimed at constant-time lookup: Engram frames “conditional memory” as a scalable lookup module grounded in modernized  $n$ -gram structure, designed as a new axis of sparsity that complements conditional computation such as MoE (Cheng et al., 2026; Shazeer et al., 2017; Fedus et al., 2021). Subsequent memory-augmented models include differentiable neural computers (Graves et al., 2016) and end-to-end memory networks (Sukhbaatar et al., 2015). Compared to retrieval and lookup-based memories, our goal is not to retrieve discrete items or facts, but to maintain a compact world state that can be updated and queried continuously. We also note architectures that build large, persistent internal memory states through multiscale update structure, e.g., Multigrid Neural Memory (Huynh et al., 2019).

Fixed-size latent alternatives also exist, where large inputs are distilled into a smaller latent workspace. Perceiver IO uses iterative attention into a fixed latent array to scale to large inputs and outputs, but the latents are typically unstructured and do not impose an explicit spatial coordinate system (Jaegle et al., 2021). In parallel, state-space sequence models such as Mamba replace attention with a recurrent state that scales linearly with sequence length, but the state is implicit and primarily designed for 1D sequences (Gu & Dao, 2023). Tensor Memory is closest in spirit to the

compact-state idea, but differs by making the state explicit and spatial, using a 3D tensor that tokens address with continuous coordinates.

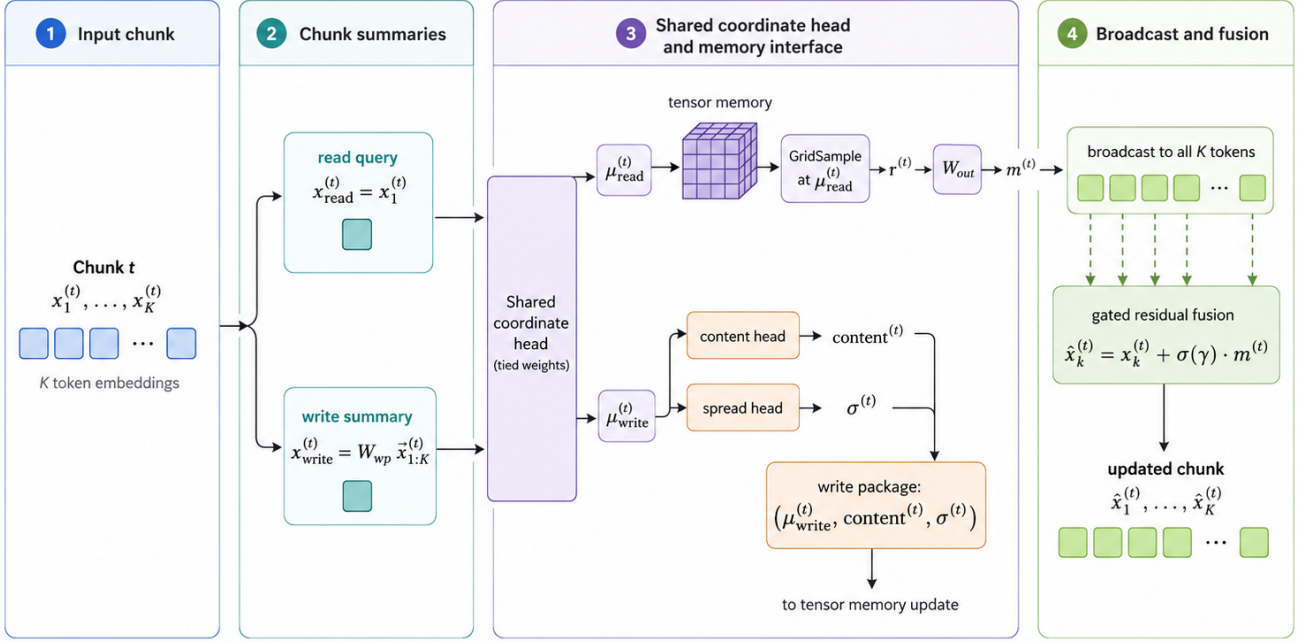
Finally, spatially structured memory appears in embodied and 3D settings. Neural Map uses a spatial memory for reinforcement learning agents to store information over long time lags (Parisotto & Salakhutdinov, 2017), and Neural SLAM learns a differentiable mapping-style external memory for exploration (Zhang et al., 2017). In vision, persistent 3D feature grids such as DeepVoxels represent scenes with a learned 3D grid of features, highlighting the value of stable volumetric representations (Sitzmann et al., 2018). More recently, M3 stores foundation-model features in a 3D Gaussian structure for spatial querying in static scenes reconstructed from video (Zou et al., 2025). While M3 focuses on building a scene-level feature field, Tensor Memory is a fixed-size recurrent voxel state attached to Transformer blocks and trained end-to-end to support long-horizon sequence tasks. Related continuous-field computation models also study local read/write heads over spatial fields (Malhotra & Seghouani, 2025), but our focus is integrating a persistent voxel state as a learnable memory inside Transformer blocks.

### 3. Model Architecture

Tensor Memory augments a Transformer with a persistent 3D voxel state. The memory is an LSTM-style (Hochreiter & Schmidhuber, 1997; Shi et al., 2015) pair of volumes

$$h_t, c_t \in \mathbb{R}^{B \times C \times D \times H \times W}. \quad (1)$$

The state is updated by scanning over chunked token groups. Figure 2 shows the per-chunk processing pipeline: chunk summaries feed a shared coordinate head, the read coordinate samples the tensor memory, and the readout is broadcast back to the chunk via a gated residual; the write package drives a separate voxel update. Figure 3 illustrates this dynamic on three concrete chunks, showing how predicted write coordinates and the accumulated memory norm  $\|h_t\|_2$  evolve over time. The remainder of this section formalizes the corresponding read/write operations, memory projection and fusion, and recurrent update. At each scan step  $t$ , the module (i) reads from  $h_t$  at a continuous coordinate, (ii) projects the retrieved memory feature back to the token dimension and fuses it into the chunk through a gated residual path, (iii) constructs a dense Gaussian write volume, (iv) computes update gates with a lightweight factorized 3D operator (Howard et al., 2017; Chollet, 2017), and (v) applies a voxel-wise gated update to obtain  $(h_{t+1}, c_{t+1})$ .



**Figure 2. Tensor Memory module, end-to-end per chunk.** (1) The chunk  $t$  consists of  $K$  token embeddings  $x_1^{(t)}, \dots, x_K^{(t)}$ . (2) We form two summaries of the chunk: a *read query*  $x_{\text{read}}^{(t)} = x_1^{(t)}$  (first token) and a *write summary*  $x_{\text{write}}^{(t)} = W_{\text{wp}} \tilde{x}_{1:K}^{(t)}$  (a learned projection over the whole chunk). (3) A shared coordinate head with tied weights produces the read coordinate  $\mu_{\text{read}}^{(t)}$  and the write coordinate  $\mu_{\text{write}}^{(t)}$ . The read coordinate samples the tensor memory via `GridSample`, returning  $r^{(t)}$ , which is projected by  $W_{\text{out}}$  to the token dimension to give the readout  $m^{(t)}$ . The write coordinate is paired with a content head (predicting  $\text{content}^{(t)}$ ) and a spread head (predicting  $\sigma^{(t)}$ ); together the write package  $(\mu_{\text{write}}^{(t)}, \text{content}^{(t)}, \sigma^{(t)})$  drives the voxel update. (4) The readout  $m^{(t)}$  is broadcast to all  $K$  tokens of the chunk and fused via a learned gated residual  $\hat{x}_k^{(t)} = x_k^{(t)} + \sigma(\gamma) m^{(t)}$ , producing the updated chunk.

### 3.1. Chunking and step inputs

Given token features  $x \in \mathbb{R}^{B \times N \times d}$ , we group tokens into chunks of size  $K$ ,

$$x_{\text{grp}} \in \mathbb{R}^{B \times S \times K \times d}, \quad S = \left\lceil \frac{N}{K} \right\rceil. \quad (2)$$

For chunk  $t$ , the read query uses the first token embedding,

$$x_{\text{read}}^{(t)} = x_{\text{grp}}[:, t, 0, :] \in \mathbb{R}^{B \times d}, \quad (3)$$

while the write signal is a learned projection of the entire chunk,

$$x_{\text{write}}^{(t)} = W_{\text{wp}} \text{vec}(x_{\text{grp}}[:, t, :, :]) \in \mathbb{R}^{B \times d}, \quad (4)$$

where  $\text{vec}(\cdot) \in \mathbb{R}^{B \times (Kd)}$  and  $W_{\text{wp}} \in \mathbb{R}^{d \times (Kd)}$ . We therefore read and write once per chunk and broadcast the resulting memory readout back to all  $K$  tokens in the chunk.

### 3.2. Shared coordinate predictor, reading, and fusion

We use a shared coordinate predictor (tied weights) for both reading and writing; the coordinates may differ because they

are computed from different chunk summaries:

$$\mu_{\text{read}}^{(t)} = \tanh(W_{\text{coord}} x_{\text{read}}^{(t)}) \in [-1, 1]^3, \quad (5)$$

$$\mu_{\text{write}}^{(t)} = \tanh(W_{\text{coord}} x_{\text{write}}^{(t)}) \in [-1, 1]^3. \quad (6)$$

Coordinates follow the `grid_sample` convention  $(x, y, z)$  corresponding to  $(W, H, D)$ . We read one point per chunk with trilinear interpolation (Jaderberg et al., 2015):

$$r^{(t)} = \text{GridSample}(h_t, \mu_{\text{read}}^{(t)}) \in \mathbb{R}^{B \times C}, \quad (7)$$

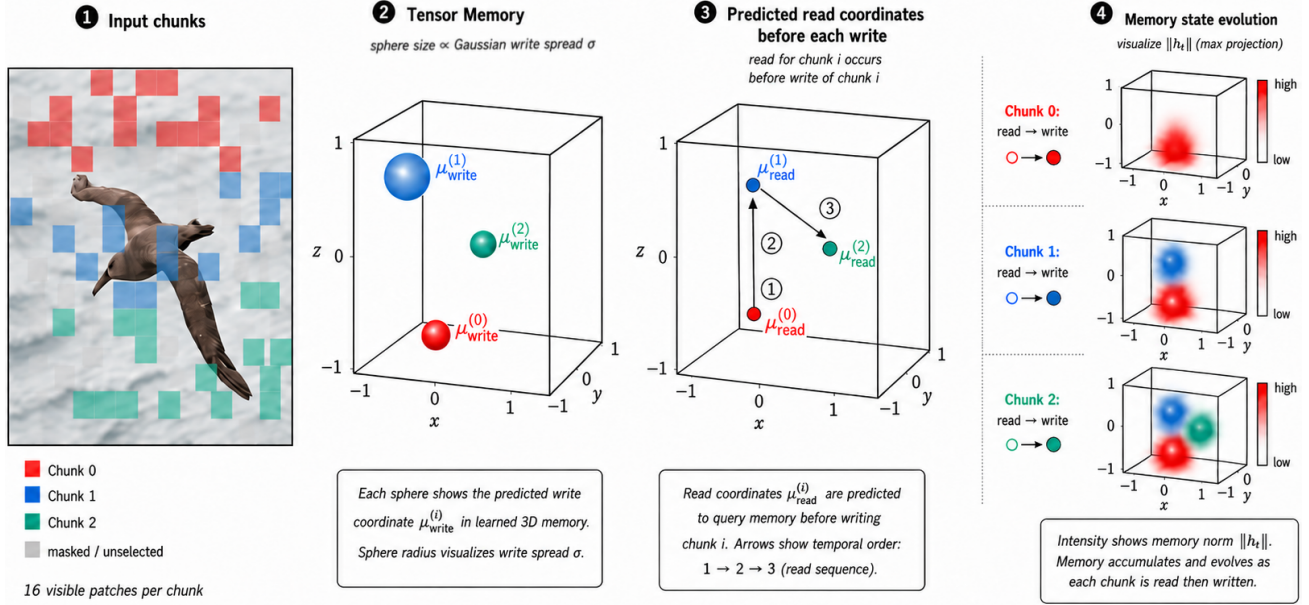
where the singleton spatial output is squeezed to obtain  $\mathbb{R}^{B \times C}$ . The raw read vector is then projected from the memory channel dimension back to the token dimension:

$$m^{(t)} = W_{\text{out}} r^{(t)} \in \mathbb{R}^{B \times d}. \quad (8)$$

Finally, the projected memory readout is broadcast to every token in the chunk and injected through a gated residual connection:

$$\hat{x}_k^{(t)} = x_k^{(t)} + \sigma(\gamma) \mathcal{D}(m^{(t)}), \quad k = 1, \dots, K, \quad (9)$$

where  $\gamma$  is a learned scalar and  $\mathcal{D}(\cdot)$  is a residual-branch regularizer. The model can increase or decrease  $\gamma$  during



**Figure 3. Tensor Memory in action, illustrated on three input chunks.** (1) The input is partitioned into chunks; here, each chunk is a small group of patches over a single frame, drawn in a distinct colour. (2) For each chunk  $i$ , the write head predicts a continuous 3D coordinate  $\mu_{\text{write}}^{(i)} \in [-1, 1]^3$  in the voxel memory; sphere radius is proportional to the predicted Gaussian write spread  $\sigma^{(i)}$ . (3) Before each write, the model reads at a predicted query coordinate  $\mu_{\text{read}}^{(i)}$ . The read for chunk  $i$  queries the memory state produced by chunks  $0, \dots, i-1$ ; arrows show the read sequence  $1 \rightarrow 2 \rightarrow 3$ . (4) Max-intensity projection of the per-voxel memory norm  $\|h_t\|_2$  after each chunk: writes accumulate into spatially structured deposits that persist across steps, giving the model an explicit, fixed-size belief state.

training; tasks that do not require memory drive  $\gamma$  negative, suppressing the memory path, while tasks that benefit from persistent state drive  $\gamma$  positive. This is compatible with standard Xavier/He weight initialization on the rest of the network and does not require a special warmup schedule. In practice we instantiate  $\mathcal{D}$  as standard dropout in the language-model experiments and as stochastic depth (DropPath) in the image and video experiments, and we initialize  $\gamma = 0$  for language modeling (so  $\sigma(\gamma) = 0.5$ ) and  $\gamma = -2$  for image and video (so  $\sigma(\gamma) \approx 0.12$ , a deliberately conservative start that lets attention dominate early in training).

### 3.3. Gaussian write volume

From  $x_{\text{write}}^{(t)}$  we predict a content vector and a positive spread  $\sigma^{(t)}$ :

$$\text{content}^{(t)} = W_c x_{\text{write}}^{(t)} \in \mathbb{R}^{B \times C}, \quad (10)$$

$$\sigma^{(t)} = \alpha \cdot \left( \text{softplus}(W_\sigma x_{\text{write}}^{(t)} + b_\sigma) + 10^{-4} \right) \in \mathbb{R}^{B \times 1} \quad (11)$$

where  $\alpha$  is a fixed scale factor (sigma\_scale in code) and  $b_\sigma$  is initialized to 1 so that the early training spread sits well above the safety floor and avoids collapse to a near-zero Gaussian. Given a cached coordinate grid  $\mathcal{G} \in \mathbb{R}^{1 \times 3 \times D \times H \times W}$  and  $\mu_{\text{write}}^{(t)}$  reshaped to  $\mathbb{R}^{B \times 3 \times 1 \times 1 \times 1}$ , we

form a dense Gaussian mask and write volume:

$$M^{(t)} = \exp \left( - \frac{\|\mathcal{G} - \mu_{\text{write}}^{(t)}\|_2^2}{2(\sigma^{(t)})^2 + \varepsilon} \right) \in \mathbb{R}^{B \times 1 \times D \times H \times W}, \quad (12)$$

$$U^{(t)} = \text{content}^{(t)} \otimes M^{(t)} \in \mathbb{R}^{B \times C \times D \times H \times W}, \quad (13)$$

where  $\otimes$  denotes broadcast multiplication and  $\varepsilon = 10^{-6}$  prevents division by zero. In our implementation,  $U^{(t)}$  is concatenated with the current state and passed to the dynamics module; we do not apply a masked in-place update to  $h_t$ .

### 3.4. Voxel-wise gated update (ConvLSTM)

We concatenate the injected write volume with the current hidden state:

$$Z^{(t)} = [U^{(t)}; h_t] \in \mathbb{R}^{B \times 2C \times D \times H \times W}. \quad (14)$$

The factorized 3D operator maps this  $2C$ -channel input to the four ConvLSTM gate tensors:

$$G^{(t)} = \text{Phys}(Z^{(t)}) \in \mathbb{R}^{B \times 4C \times D \times H \times W}. \quad (15)$$

In our implementation, Phys is a depthwise-separable 3D operator formed by three axis-factorized depthwise convolutions followed by a pointwise projection:

$$\text{Phys}(z) = \text{Conv}_{1 \times 1 \times 1}^{2C \rightarrow 4C} (\text{DW}^{1 \times 1 \times 3} (\text{DW}^{1 \times 3 \times 1} (\text{DW}^{3 \times 1 \times 1} (z)))) \quad (16)$$

where  $DW^k$  denotes a depthwise convolution with kernel  $k$  and same padding. We split  $G^{(t)}$  into four  $C$ -channel tensors and compute ConvLSTM-style gates:

$$[i_t, f_t, o_t, g_t] = \text{chunk}(G^{(t)}, 4), \quad (17)$$

$$i_t = \sigma(i_t), \quad (18)$$

$$f_t = \sigma(f_t), \quad (19)$$

$$o_t = \sigma(o_t), \quad (20)$$

$$g_t = \tanh(g_t), \quad (21)$$

$$c_{t+1} = f_t \odot c_t + i_t \odot g_t, \quad (22)$$

$$h_{t+1} = o_t \odot \tanh(c_{t+1}), \quad (23)$$

where all operations are voxel-wise over  $(D, H, W)$ .

## 4. Experiments

We evaluate Tensor Memory on three standard benchmarks across language, images, and video, and on a controlled toy diagnostic suite designed to isolate *when* a fixed-size recurrent 3D state is beneficial (Tables 1 to 4). In every setting we use the same backbone for the compared methods and vary only the memory mechanism, keeping all other hyperparameters fixed. Tensor Memory yields the largest absolute gains on language, where the persistent state accumulates context over a 512-token sequence; gains are smaller but consistent on image patch reconstruction; on UCF-101 it outperforms both the joint Transformer baseline and a strong register-token baseline on Top-1 and Top-5 accuracy, at the cost of substantially higher training wall-clock from the per-step voxel recurrence. Full toy task definitions and the sweep grid are in Appendix A.

**Language modeling on WikiText-2.** We train a decoder-only Transformer from scratch on WikiText-2 (Merity et al., 2016) with  $d=384$ , 8 heads, 8 layers, batch size 32, sequence length 512, cosine schedule with 2000 warmup steps, and dropout 0.2. We compare **Base** (full-attention Transformer, the primary baseline), **Base+SLN** (Base + post-attention LayerNorm), **Local** (windowed attention with window 64), **Transformer-XL** (segment-level recurrence (Dai et al., 2019)), **Linear** (ELU+1 linear attention), and **Tensor Memory** (ours, same backbone augmented with the 3D voxel state); all share the same backbone and training budget (40K steps with early stopping). We report validation and test perplexity (PPL $\downarrow$ ) in Table 1.

**Image patch reconstruction on CUB-200-2011.** We evaluate a masked-patch reconstruction task on CUB-200-2011 (Wah et al., 2011) using a ViT-style encoder trained from scratch: the model receives a randomly masked image and must reconstruct the missing patches. We compare four variants: **Base ViT** (full-attention, primary baseline), **Local ViT** (windowed attention), **Registers ViT** (ViT aug-

| Method                      | Params       | Val PPL $\downarrow$ | Test PPL $\downarrow$ |
|-----------------------------|--------------|----------------------|-----------------------|
| Base                        | 25.9M        | 138.91               | 130.30                |
| Base + SLN                  | 25.9M        | 143.32               | 135.74                |
| Local                       | 25.9M        | 164.23               | 153.57                |
| Transformer-XL              | 25.7M        | 152.55               | 144.05                |
| Linear                      | 25.9M        | 164.27               | 153.38                |
| <b>Tensor Memory (ours)</b> | <b>28.3M</b> | <b>106.10</b>        | <b>100.07</b>         |

Table 1. Language modeling on WikiText-2. All models: decoder-only Transformer,  $d=384$ , 8 heads, 8 layers, seq=512, trained for up to 40K steps with early stopping. *Base* is the primary baseline; *Base+SLN* adds a post-attention LayerNorm; *Local* uses a window of 64; *Transformer-XL* uses segment-level recurrence (Dai et al., 2019); *Linear* is an ELU+1 linear-attention variant.

| Method                    | Params       | PSNR (25/50/75%) $\uparrow$ | SSIM (25/50/75%) $\uparrow$ |
|---------------------------|--------------|-----------------------------|-----------------------------|
| Base ViT                  | 2.30M        | 24.73/21.77/20.07           | 0.859/0.728/0.606           |
| Local ViT                 | 2.30M        | 24.79/21.80/20.03           | 0.857/0.726/0.605           |
| Registers ViT             | 2.30M        | 24.71/21.75/20.08           | 0.859/0.729/0.606           |
| <b>Spatial ViT (ours)</b> | <b>2.90M</b> | <b>24.85/21.90/20.19</b>    | <b>0.861/0.732/0.609</b>    |

Table 2. Masked patch reconstruction on CUB-200-2011 (Wah et al., 2011). PSNR $\uparrow$  and SSIM $\uparrow$  at mask ratios 25/50/75%, evaluated at test time. All variants share the same encoder depth and hidden dimension. *Base ViT* is full-attention (primary baseline); *Local ViT* uses windowed attention; *Registers ViT* adds 8 learnable global tokens; *Spatial ViT* adds Tensor Memory.

mented with 8 learnable global tokens), and **Spatial ViT** (ours, Tensor Memory). ViT overfits to texture on fine-grained datasets; the reconstruction task focuses evaluation on structural rather than textural fidelity, which better reflects the spatial aggregation capability we claim. We report PSNR $\uparrow$  and SSIM $\uparrow$  at mask ratios 25%, 50%, and 75% in Table 2.

**Video action recognition on UCF-101.** We train a video Transformer from scratch on UCF-101 (Soomro et al., 2012) with  $d=256$ , 6 layers, 8 heads, and  $T=8$  frames per clip (no pretrained backbone). We compare **Baseline** (joint spatiotemporal Transformer), **Registers** (Baseline augmented with  $R=8$  persistent register tokens), and **Spatial** (ours, Tensor Memory); all train for 50 epochs with identical settings. We report Top-1 and Top-5 accuracy on the UCF-101 test split in Table 3.

**Toy diagnostics.** We complement the benchmarks with a controlled toy suite designed to isolate when a fixed-size recurrent 3D state helps over sequence-only Transformers. Each task forces the model to maintain information that disappears from the current token stream (occlusion, partial observability, or long retrieval gaps); a no-harm control deliberately requires no persistent state. We compare four backbones at matched depth and width: *Base* (standard Trans-

| Method                | Params | Top-1↑        | Top-5↑        | Train      |
|-----------------------|--------|---------------|---------------|------------|
| Baseline              | 5.01M  | 82.12%        | 93.22%        | 84.0 min   |
| Registers ( $R=8$ )   | 5.01M  | 88.34%        | 94.93%        | 84.9 min   |
| <b>Spatial (ours)</b> | 5.61M  | <b>89.21%</b> | <b>95.98%</b> | 1331.4 min |

Table 3. Video action recognition on UCF-101 (Soomro et al., 2012). All models trained from scratch ( $d=256$ , 6 layers, 8 heads,  $T=8$  frames, 50 epochs, no pretrained backbone). *Baseline* is a standard joint spatiotemporal Transformer; *Registers* adds  $R=8$  persistent learnable global tokens; *Spatial* adds Tensor Memory. “Train” is total training wall-clock on a single GPU; the per-step recurrent scan over the 3D voxel state is currently the throughput bottleneck.

| Toy                                          | Metric | Base  | Base wide | Slots | Tensor       |
|----------------------------------------------|--------|-------|-----------|-------|--------------|
| Coordinate binding ( $W=20$ , $\sigma=0.1$ ) | Acc.↑  | 90.8% | 91.0%     | 90.4% | <b>94.8%</b> |
| No-harm (seq=32)                             | Acc.↑  | 100%  | 100%      | 100%  | 100%         |
| Occlusion ( $L=4$ )                          | Acc.↑  | 100%  | 99.5%     | 100%  | <b>100%</b>  |
| Map building ( $T=32$ )                      | Acc.↑  | 50.6% | 51.0%     | 50.2% | <b>58.5%</b> |

Table 4. Toy diagnostics: query/classification accuracy at one representative difficulty per task. **Coordinate binding**: predict the value of the nearest written coordinate from  $W$  writes under query noise  $\sigma$ ; Tensor Memory’s advantage grows sharply with  $W$  – at  $W=100$  all attention-based baselines collapse to near-random (7–14%) while Tensor remains at 75–91% across noise levels (Appendix A). **No-harm**: a short fully-observed copy/shift task where memory should not help; all methods reach 100% and the learned tensor gate  $\sigma(\gamma)$  stays at its 0.5 initialisation (mean 0.505), confirming the memory path is suppressed when not useful. **Occlusion**: a ball is hidden by an occluder for  $L$  middle frames; predict the final-frame quadrant. All methods are perfect at  $L=4$  and all collapse together once  $L \geq 8$  (the task’s information limit), so occlusion serves as a no-regression check rather than a separation result. **Map building**: agent observes a  $2 \times 2$  patch of an  $8 \times 8$  binary grid for  $T$  steps and is then queried at one cell; baselines never escape chance ( $\approx 50$ –55%) at any horizon  $T \in \{8, \dots, 128\}$ , while Tensor Memory reaches 61.9% at  $T=64$ . Full sweeps and learning curves appear in Appendix A.

former), *Base wide* (wider MLP, parameter-matched to *Tensor*), *Slots* (Transformer with 8 persistent learnable global tokens), and *Tensor* (ours). **Tensor Memory matches or beats every baseline on every toy**. The advantage is largest on coordinate binding and grows sharply with the number of writes: at  $W=100$  all attention-based baselines collapse to near-random (7–14%) while Tensor Memory remains at 75–91% across noise levels (Appendix A). On the no-harm control all methods reach 100% and the learned tensor gate  $\sigma(\gamma)$  stays at its 0.5 initialisation (mean 0.505), confirming that the memory path is suppressed when not useful. Table 4 reports a single representative configuration per task; full sweeps and ablations are in Appendix A.

**What can be inspected at inference time.** Tensor Memory is straightforward to inspect without any extra training objectives, simply by logging internal tensors. The write

head exposes the continuous write coordinate  $\mu_{\text{write}}^{(t)}$ , the predicted spread  $\sigma^{(t)}$ , and the dense Gaussian mask  $M^{(t)}$  over the voxel grid; the hidden state  $h$  can be summarised as a per-voxel feature-norm volume  $\|h\|_2$  and rendered via orthogonal slices or a maximum-intensity projection; the read coordinate  $\mu_{\text{read}}^{(t)}$  traces a query trajectory across steps; and the learned scalar gate  $\sigma(\gamma)$  provides a single number per layer indicating how much the model is currently using memory. We treat the memory as a 3D voxel grid throughout, but the same write/read/update mechanism generalises to memory tensors of arbitrary dimension (e.g., 4D for a time-aware spatial memory, or higher-D for memories that include a learned camera-pose or modality axis); we discuss this further in the next section.

## 5. Discussion and Future Work

Tensor Memory is a step toward Transformer models that maintain an explicit, compact state rather than relying on an ever-growing token history. Our results suggest that a persistent structured memory can complement attention, especially in settings where the input stream is partial, occluded, or simply too long to keep in full resolution. The current design makes several choices that are worth discussing.

### Where Tensor Memory helps and where it may not.

The biggest absolute gains appear when the task benefits from maintaining state across long horizons (e.g., long-context language); on shorter-horizon tasks the gains are typically smaller. When the memory path is genuinely not useful, the learned scalar gate  $\sigma(\gamma)$  provides an additional control to suppress it: in our no-harm diagnostic the gate stays near its initialization rather than ramping up. A separate limitation is that most of our experiments use single-step supervision while many downstream uses are autoregressive; multi-step rollout objectives (optionally with scheduled sampling) would directly test robustness under distribution shift. A fixed-size state can also become a bottleneck if the task truly requires storing many distinct details, in which case performance may saturate with memory resolution or channel width.

**Higher-dimensional memory.** We treat the memory as a 3D voxel grid throughout this work, but the write/read/update mechanism generalises to memory tensors of arbitrary dimension. A 4D memory could index a (time,  $x, y, z$ ) axis to make the temporal scan explicit, and higher-D memories could include a learned camera-pose axis or a modality channel. We restricted ourselves to 3D here primarily because higher-dimensional grids are expensive to train end-to-end at resolutions that would be informative on the benchmarks we considered; combining sparser updates, hierarchical grids, or further factorisation (analogous to our factorised 3D convolution) would make

this practical. We view this as one of the most promising directions for future work.

**Architecture and dynamics.** Our addressing uses continuous coordinates with Gaussian-weighted writes, which makes optimisation stable and avoids hard assignment but raises questions about how addressing should be constrained. Future work could explore sharper or adaptive kernels, multi-scale writes, sparsified writes to reduce compute, or learned coordinate frames that better match camera motion in video. Richer write semantics (e.g., key-value writes, content-based routing, confidence-weighted updates) could let the memory represent uncertainty and handle conflicting evidence. Our dynamics module is intentionally lightweight (factorised depthwise 3D filtering plus gated recurrence); stronger but still efficient alternatives include multi-scale operators, anisotropic updates conditioned on motion, learned advection-style updates, or explicit physical priors. For video specifically, warping the memory using optical flow or camera ego-motion before each update would make persistence significantly easier.

**Scaling, evaluation, and broader impacts.** Although the memory size is constant, the current implementation does dense voxel updates at each scan step — the dominant cost at scale (cf. the UCF-101 wall-clock in Table 3). Sparse updates over a subset of voxels, low-rank or quantized memory channels, and hierarchical grids that allocate capacity adaptively are concrete paths to lower this cost. On the evaluation side, larger long-horizon video benchmarks, scaled-up versions of our occlusion and partial-observation toys (Appendix A), and transfer studies (whether a memory trained on one distribution generalises to longer horizons or different occlusion patterns) would all sharpen the claims. A compact persistent state could improve long-horizon perception by reducing sensitivity to missing frames and enabling more stable temporal reasoning; at the same time, any mechanism that increases a model’s capacity to retain information raises privacy concerns when trained on sensitive data, and this should be considered in dataset choice and deployment.

In summary, Tensor Memory suggests that adding a small structured state to Transformers can be a practical way to handle long-horizon reasoning. We view this work as a foundation: the current module is deliberately simple, and there are many straightforward paths — including higher-dimensional grids, richer dynamics, and sparse updates — to improve the memory representation, the dynamics, and the efficiency while keeping the core idea of a fixed-size, explicit state.

## 6. Conclusion

We presented Tensor Memory, a simple way to give Transformers an explicit, fixed-size 3D state that can persist over long horizons. Instead of storing an ever-growing history of tokens, the model writes information into a small voxel grid using continuous coordinates, updates the grid with a lightweight recurrent dynamics module, and reads back context through continuous sampling with a gated residual connection. This design keeps the strengths of standard Transformers while adding a stable place to store persistent scene-level information, which is especially useful when observations are partial or occluded. On WikiText-2 language modeling Tensor Memory yields the largest gains, dropping test perplexity from 130.30 to 100.07; on CUB-200 patch reconstruction it gives smaller but consistent improvements over base, local, and register-token ViTs across mask ratios. On UCF-101 it outperforms both the joint Transformer baseline and a strong register-token baseline on Top-1 and Top-5 accuracy, at the cost of substantially higher training wall-clock from the per-step voxel recurrence. On a controlled toy suite designed to isolate when persistent state helps (Appendix A), Tensor Memory matches or beats a full-attention baseline, a parameter-matched wider baseline, and a learned-slot memory baseline on every task; the advantage grows sharply with task difficulty, with attention baselines collapsing on coordinate binding at large numbers of writes while Tensor Memory remains accurate. We hope this pushes toward Transformer systems that carry compact, structured state, and that it offers a practical building block for long-horizon perception and reasoning.

## 7. Acknowledgments

We would like to thank Manel Baradad and Minyoung Huh for their helpful discussions and advice.

## References

- Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., and Sanghai, S. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.
- Beltagy, I., Peters, M. E., and Cohan, A. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- Bertasius, G., Wang, H., and Torresani, L. Is space-time attention all you need for video understanding? *arXiv preprint arXiv:2102.05095*, 2021.
- Borgeaud, S., Mensch, A., Hoffmann, J., Cai, T., Rutherford, E., Millican, K., van den Driessche, G., Lespiau, J.-B., Damoc, B., Clark, A., et al. Improving language models

- by retrieving from trillions of tokens. *arXiv preprint arXiv:2112.04426*, 2021.
- Bulatov, A., Kuratov, Y., and Burtsev, M. S. Recurrent memory transformer. *arXiv preprint arXiv:2207.06881*, 2022.
- Cheng, X., Zeng, W., Dai, D., Chen, Q., Wang, B., Xie, Z., Huang, K., Yu, X., Hao, Z., Li, Y., Zhang, H., Zhang, H., Zhao, D., and Liang, W. Conditional memory via scalable lookup: A new axis of sparsity for large language models. Technical Report arXiv:2601.07372, arXiv, 2026.
- Chollet, F. Xception: Deep learning with depthwise separable convolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- Choromanski, K., Likhoshesterov, V., Dohan, D., Song, X., Gane, A., Sarlos, T., Hawkins, P., Davis, J., Mohiuddin, A., Kaiser, L., Belanger, D., Colwell, L., and Weller, A. Rethinking attention with performers. *arXiv preprint arXiv:2009.14794*, 2020.
- Dai, Z., Yang, Z., Yang, Y., Carbonell, J., Le, Q. V., and Salakhutdinov, R. Transformer-xl: Attentive language models beyond a fixed-length context. Technical Report arXiv:1901.02860, arXiv, 2019.
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness. *arXiv preprint arXiv:2205.14135*, 2022.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., and Houshy, N. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *arXiv preprint arXiv:2101.03961*, 2021.
- Graves, A., Wayne, G., and Danihelka, I. Neural turing machines. Technical Report arXiv:1410.5401, arXiv, 2014.
- Graves, A., Wayne, G., Reynolds, M., Harley, T., Danihelka, I., Grabska-Barwińska, A., Colmenarejo, S. G., Grefenstette, E., Ramalho, T., Agapiou, J., Badia, A. P., Hermann, K. M., Zwols, Y., Ostrovski, G., Cain, A., King, H., Summerfield, C., Blunsom, P., Kavukcuoglu, K., and Hassabis, D. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538 (7626):471–476, 2016.
- Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- Hochreiter, S. and Schmidhuber, J. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., and Adam, H. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.
- Huynh, T., Maire, M., and Walter, M. R. Multigrid neural memory. *arXiv preprint arXiv:1906.05948*, 2019.
- Jaderberg, M., Simonyan, K., Zisserman, A., and Kavukcuoglu, K. Spatial transformer networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- Jaegle, A., Borgeaud, S., Alayrac, J.-B., Doersch, C., Ionescu, C., Ding, D., Koppula, S., Zoran, D., Brock, A., Shelhamer, E., Hénaff, O., Botvinick, M., Zisserman, A., Vinyals, O., and Carreira, J. Perceiver io: A general architecture for structured inputs & outputs. Technical Report arXiv:2107.14795, arXiv, 2021.
- Karami, M., Behrouz, A., Kacham, P., and Mirrokni, V. Trelis: Learning to compress key-value memory in attention models. *arXiv preprint arXiv:2512.23852*, 2025.
- Khandelwal, U., Levy, O., Jurafsky, D., Zettlemoyer, L., and Lewis, M. Generalization through memorization: Nearest neighbor language models. *arXiv preprint arXiv:1911.00172*, 2019.
- Kitaev, N., Kaiser, L., and Levskaya, A. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.
- Kwon, W., Li, Z., Zhuang, C., Sheng, Y., Zheng, L., Yu, C., Gonzalez, J. E., and Stoica, I. Efficient memory management for large language model serving with page-dattention. *arXiv preprint arXiv:2309.06180*, 2023.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Stoyanov, V., and Riedel, S. Retrieval-augmented generation for knowledge-intensive nlp tasks. *arXiv preprint arXiv:2005.11401*, 2020.
- Liu, A. et al. Deepseek-v3 technical report. Technical Report arXiv:2412.19437, arXiv, 2024.
- Liu, Z., Ning, J., Cao, Y., Wei, Y., Zhang, Z., Lin, S., and Hu, H. Video swin transformer. *arXiv preprint arXiv:2106.13230*, 2021.
- Malhotra, A. and Seghouani, N. Neural field turing machine: A differentiable spatial computer. *arXiv preprint arXiv:2509.03370*, 2025.

- Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. In *International Conference on Learning Representations (ICLR)*, 2016. URL <https://arxiv.org/abs/1609.07843>.
- Munkhdalai, T., Faruqui, M., and Gopal, S. Leave no context behind: Efficient infinite context transformers with infini-attention. *arXiv preprint arXiv:2404.07143*, 2024.
- Parisotto, E. and Salakhutdinov, R. Neural map: Structured memory for deep reinforcement learning. Technical Report arXiv:1702.08360, arXiv, 2017.
- Rae, J. W., Potapenko, A., Jayakumar, S. M., and Lillicrap, T. P. Compressive transformers for long-range sequence modelling. Technical Report arXiv:1911.05507, arXiv, 2019.
- Shazeer, N. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., and Hinton, G. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- Shi, X., Chen, Z., Wang, H., Yeung, D.-Y., Wong, W.-k., and Woo, W.-c. Convolutional lstm network: A machine learning approach for precipitation nowcasting. *arXiv preprint arXiv:1506.04214*, 2015.
- Sitzmann, V., Thies, J., Heide, F., Nießner, M., Wetzstein, G., and Zollhöfer, M. Deepvoxels: Learning persistent 3d feature embeddings. Technical Report arXiv:1812.01024, arXiv, 2018.
- Soomro, K., Zamir, A. R., and Shah, M. Ucf101: A dataset of 101 human actions classes from videos in the wild. *arXiv preprint arXiv:1212.0402*, 2012.
- Sukhbaatar, S., Weston, J., Fergus, R., et al. End-to-end memory networks. *arXiv preprint arXiv:1503.08895*, 2015.
- Touvron, H., Cord, M., Douze, M., Massa, F., Sablayrolles, A., and Jégou, H. Training data-efficient image transformers & distillation through attention. *arXiv preprint arXiv:2012.12877*, 2020.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Wah, C., Branson, S., Welinder, P., Perona, P., and Belongie, S. The caltech-ucsd birds-200-2011 dataset. Technical report, California Institute of Technology, 2011.
- Wang, S., Li, B. Z., Khabsa, M., Fang, H., and Ma, H. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.
- Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.
- Zaheer, M., Guruganesh, G., Dubey, A., Ainslie, J., Alberti, C., Ontañón, S., Pham, P., Ravula, A., Wang, Q., Yang, L., and Ahmed, A. Big bird: Transformers for longer sequences. *arXiv preprint arXiv:2007.14062*, 2020.
- Zhang, J., Tai, L., Liu, M., Boedecker, J., and Burgard, W. Neural slam: Learning to explore with external memory. *arXiv preprint arXiv:1706.09520*, 2017.
- Zou, Z. et al. M3: 3d-spatial multimodal memory. *arXiv preprint arXiv:2503.16413*, 2025.

## A. Toy Diagnostics

We ran a controlled toy suite to isolate when a fixed-size recurrent 3D state helps over sequence-only Transformers (Table 5). Each task forces the model to maintain information that disappears from the current token stream (occlusion, partial observability, or long retrieval gaps); a no-harm control deliberately requires no persistent state. **Tensor Memory matches or beats every baseline on every task.** The gap is largest on coordinate binding and grows sharply with the number of writes  $W$ : at  $W=5$ –20 all four methods are within a few points of each other, but at  $W=100$  the attention-based baselines (Base, Base-wide, Slots) collapse to 7–14% accuracy while Tensor Memory remains at 75–91% across noise levels, and at  $W=200$  the baselines drop to near-random ( $\approx 3\%$ ) while Tensor Memory still maintains 55–72%. On the no-harm control all methods reach 100% and the learned tensor gate  $\sigma(\gamma)$  stays at its 0.5 initialisation (mean 0.505), confirming the memory path is suppressed when not useful. A summary at one representative configuration per task appears in Table 4 (main paper); reference implementations of all four tasks plus the ablation suite are included with the code release.

**Baselines.** We compared against a standard Transformer with the same backbone (*Base*), a parameter-matched variant with a wider MLP (*Base-wide*, which separates “more params” from “better memory”), and a fixed-slot memory baseline (*Slots*: 8 learnable global tokens read/written via attention).

**Toy 1: Occlusion and object permanence.** Short videos of a single bouncing ball partially hidden by a static rectangular occluder for a variable number of consecutive frames. The model predicts the ball’s final-frame quadrant. We sweep the occlusion length  $L$  and report accuracy as a function of  $L$ .

**Toy 2: Long-horizon partial observations (map building).** At each step the model observes a small  $2 \times 2$  patch of a hidden  $8 \times 8$  binary occupancy grid (with the patch position included as part of the observation token); at the end of  $T$  steps it is queried about one cell. We sweep the horizon  $T$  and report query accuracy.

**Toy 3: Addressing and binding.** Tokens write values to continuous 3D coordinates; later tokens query those coordinates under additive Gaussian noise of standard deviation  $\sigma$ . We sweep the number of writes  $W$  and the query noise  $\sigma$  and report retrieval accuracy, directly testing whether Tensor Memory learns stable coordinate binding.

**Toy 4: Control (no-harm).** A short, fully-observed copy/shift task where persistent state should not help. We verify that Tensor Memory does not deteriorate relative to the baselines and report learned gate statistics to confirm that the memory branch stays near-silent.

**Toy ablations.** We ablated, all on Toy 3 at  $W=20$ ,  $\sigma=0.05$ : Gaussian-weighted write vs. hard nearest-voxel assignment; shared vs. separate read/write coordinate heads; the factorized 3D physics operator on vs. off (replaced by a  $1 \times 1 \times 1$  pointwise gate); memory resolution  $4^3/6^3/8^3$ ; and chunk size  $1/2/4$ .

| Toy                | Setup                                                     | What is hidden / missing?                                        | What we sweep                               | Primary plot / metric                                             |
|--------------------|-----------------------------------------------------------|------------------------------------------------------------------|---------------------------------------------|-------------------------------------------------------------------|
| Occlusion tracking | Short videos with a moving ball and a static occluder     | Ball is unobserved during the occluded frames                    | Occlusion length $L$                        | Accuracy vs. $L$                                                  |
| Map building       | Partial $2 \times 2$ observations of an $8 \times 8$ grid | Most cells are never visible at any single step                  | Horizon $T$                                 | Query accuracy vs. $T$                                            |
| Coordinate binding | Stream of write tokens followed by noisy query tokens     | Values must be retrieved from continuous coordinates under noise | Number of writes $W$ , query noise $\sigma$ | Retrieval accuracy vs. $W$ at fixed $\sigma$                      |
| No-harm control    | Short, fully-observed copy/shift sequence                 | Nothing (memory should be unnecessary)                           | —                                           | Accuracy near baseline; learned gate $\sigma(\gamma) \approx 0.5$ |

Table 5. Toy diagnostics. The first three toys each stress a different aspect of persistent state (occlusion, limited field-of-view, delayed retrieval); the no-harm control deliberately requires no persistent state and verifies that the memory path can be suppressed. Tensor Memory matched or beat every baseline on every toy; numerical results at one representative configuration per toy are reported in Tab. 4 (main paper).